ΕΝΔΙΑΜΕΣΗ ΕΞΕΤΑΣΗ Τρίτη, 27/10/2025, ώρα διάλεξης

Σε δύο αίθουσες (αίθουσα διάλεξης και 147), θα ανακοινωθεί λίγο πριν την εξέταση σε ποιο χώρο θα πάτε

Είσοδος και Έξοδος

Input and Output

Είσοδος και Έξοδος

- **Ορίσματα από γραμμή εντολής**
  - Command line arguments
- **Διαδραστικά**
  - Interactively with user
- **Με ανακατεύθυνση**
  - File redirection
- **Σωλήνωση διεργασιών**
  - Process piping
- **Με ρητά αναφορά σε αρχεία**
  - File Input/Output
- **Απλά γραφικά για visual output**

Standard Input/Output

- Standard Input channel (κανάλι εισόδου)
  - Πληκτρολόγιο
- Standard Output channel (κανάλι εξόδου)
  - Οθόνη

Είσοδος και Έξοδος

Μονάδες Εισόδου

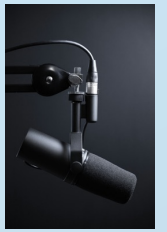

Πληκτρολόγιο


Ποντίκι


Σκληρός δίσκος


Διαδίκτυο


Ψηφιακή κάμερα


Μικρόφωνο

Μονάδες Εξόδου


Οθόνη


Ηχεία


Σκληρός δίσκος


Διαδίκτυο


Εκτυπωτής

Στόχος: Τα προγράμματα μας σε Java να επικοινωνούν με τον έξω κόσμο

Ορίσματα από γραμμή εντολής

- Διοχέτευση δεδομένων προς τη συνάρτηση main

```
public static void main(String[] args){…}
```

- Διασύνδεση προγράμματος με λειτουργικό σύστημα
- Για μικρό, αλλά και μεταβλητό, πλήθος δεδομένων
  - `args.length` δίνει το τρέχον πλήθος
- Τα δεδομένα εισάγονται ως strings και επομένως χρειάζεται να μετασχηματιστούν στους σχετικούς βασικούς τύπους δεδομένων
  - Wrapper classes

```
// Computing the average of a list of numbers
// Version 1: Input from command line

public class Average {

 public static void main(String[] args) {
  double sum = 0.0;
  int cnt = 0;
  for (String s : args) {
     sum += Double.parseDouble(s);
     cnt++;
  }
  double average = sum/cnt;
  System.out.println("Average is "
 }

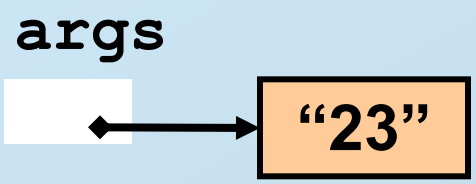
}
```

```
$ java Average 23
Average is 23.0

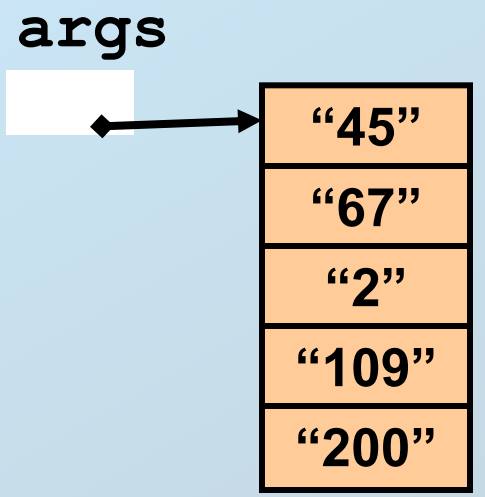
$ java Average 45 67 2 109 200
Average is 84.6

$java Average
Average is NaN
```

```
$ java Average 23
Average is 23.0
```



```
$ java Average 45 67 2 109 200
Average is 84.6
```



```
$java Average
Average is NaN
```



`args.length == 0`

```java
// Reversing the list of arguments from command line

public class ReverseArgs {

 public static void main(String[] args) {
   if (args.length == 0)
      System.out.print("Nothing to reverse ...");
   for (int i = args.length - 1; i >= 0; i--)
      System.out.print(args[i] + "  ");
   System.out.print("\n\n");
 }
}
```

```
$ java ReverseArgs one two three
three  two  one

$ java ReverseArgs This is a working day
day  working  a  is  This

$ java ReverseArgs
Nothing to reverse ...
```

Μορφοποίηση Εξόδου (formatting) printf

```
public class Average {

 public static void main(String[] args) {
  double sum = 0.0;
  if (args.length == 0)
     System.out.print("Nothing to average ...");
  else {
     for (String s : args) {
        sum += Double.parseDouble(s);
     }
     double average = sum/args.length;
     System.out.printf("Average is %8.4f\n",average);
   }
 }
}
```

String μορφοποίησης
% specifier ορισμός

Λιστά εκφράσεων εξόδου

format string specifier

%8.4f

field width

precision

conversion code

```
$ java Average 45 67 2 109 200
Average is  84.6000
```

8 4

Μορφοποίηση Εξόδου (formatting) printf

type	*code*	*typical literal*	*sample format strings*	*converted string values for output*
int	d	512	"%14d" "%-14d"	" 512" "512 "
double	f e	1595.1680010754388	"%14.2f" "%.7f" "%14.4e"	" 1595.17" "1595.1680011" " 1.5952e+03"
String	s	"Hello, World"	"%14s" "%-14s"	" Hello, World" "Hello, World "

Διαδραστική Εισδοχή Δεδομένων (standard input)

- Standard Input channel (κανάλι εισόδου)
  - Πληκτρολόγιο την ώρα που τρέχει το πρόγραμμα
- Μέσω διαλόγου με τον χρήστη
- Κατάλληλα μηνύματα για εισδοχή
  - User prompts
- Δεν αφορά μόνο τη main αλλά οποιαδήποτε συνάρτηση
- Δεν υπάρχει περιορισμός ως προς το πλήθος των στοιχείων
  - Πρακτικοί περιορισμοί
- Βιβλιοθήκη **`StdIn`** , Scanner

Standard Είσοδος και Έξοδος

Standard είσοδος: Παρέχεται η βιβλιοθήκη `StdIn`

public class StdIn	
boolean isEmpty()	`true` *if no more values,* `false` *otherwise*
int readInt()	*read a value of type* `int`
double readDouble()	*read a value of type* `double`
long readLong()	*read a value of type long*
boolean readBoolean()	*read a value of type* `boolean`
char readChar()	*read a value of type* `char`
String readString()	*read a value of type* `String`
String readLine()	*read the rest of the line*
String readAll()	*read the rest of the text*

Standard έξοδος: Παρέχεται η βιβλιοθήκη `StdOut` (ταυτόσημη με τη βιβλιοθήκη `System.out`)

public class StdOut	
void print(String s)	*print* `s`
void println(String s)	*print* `s`*, followed by newline*
void println()	*print a new line*
void printf(String f, ...)	*formatted print*

Στατικές Μέθοδοι – public static

Standard Είσοδος with Scanner

```
import java.util.Scanner;
Scanner inp = Scanner(System.in);
```

```
boolean hasNext()

int     nextInt()

double nextDouble()

long    nextLong()

Boolean nextBoolean()

String nextString()

String nextLine()
```

```
// Computing the average of a list of numbers
// Version 2: Input entered interactively

public class Average {

 public static void main(String[] args) {
  double sum = 0.0;
  int cnt = 0;
  System.out.print("Enter value: ");
  while (!StdIn.isEmpty()) {
     double value = StdIn.readDouble();
     sum += value;
     cnt++;
     System.out.print("Enter value: ");
  }
  double average = sum/cnt;
  System.out.println("\nAverage is " + average);
 }

}
```

```
// Computing the average of a list of numbers
// Version 2: Input entered interactively

public class Average {

 public static void main(String[] args) {
  double sum = 0.0;
  int cnt = 0;
  System.out.print("Enter value: ");
  while (!StdIn.isEmpty()) {
     sum +=  StdIn.readDouble();
     cnt++;
     System.out.print("Enter value: ");
  }
  double average = sum/cnt;
  System.out.println("\nAverage is " + average);
 }

}
```

```
$ java Average
Enter value: 23
Enter value: <Ctrl-d> or <Ctrl-Z, Enter>
Average is 23.0

$ java Average
Enter value: 45
Enter value: 67
Enter value: 2
Enter value: 109
Enter value: 200
Enter value: <Ctrl-d> or <Ctrl-Z, Enter>
Average is 84.6

$ java Average
Enter value: <Ctrl-d> or <Ctrl-Z, Enter>
Average is NaN
```

```
// Reversing a list of numbers entered interactively

public class Reverse {

 public static void rev(int i) {
   System.out.print("Enter number " + i + ": ");
   int n = StdIn.readInt();
   if (n==0) {
    System.out.println(
        "The numbers in reverse order are:");
    return; }
   rev(i+1);
   System.out.println("Number " + i + " is " + n);
 }

 public static void main(String[] args){
   System.out.println("Enter the sequence of numbers to be
reversed, terminating with 0");
   rev(1);
 }
}
```

```
$ java Reverse
Enter the sequence of numbers to be reversed,
terminating with 0
Enter number 1: 34
Enter number 2: 89
Enter number 3: 56
Enter number 4: 12
Enter number 5: 0
The numbers in reverse order are:
Number 4 is 12
Number 3 is 56
Number 2 is 89
Number 1 is 34

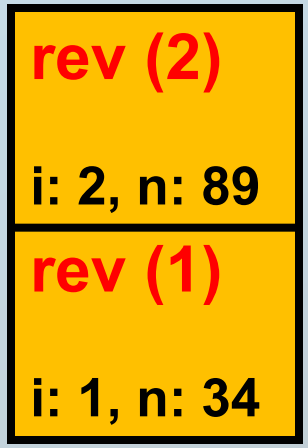
$ java Reverse
Enter the sequence of numbers to be reversed,
terminating with 0
Enter number 1: 0
The numbers in reverse order are:
```

```
$ java Reverse
Enter the sequence of numbers to be reversed,
terminating with 0
Enter number 1: 34
```

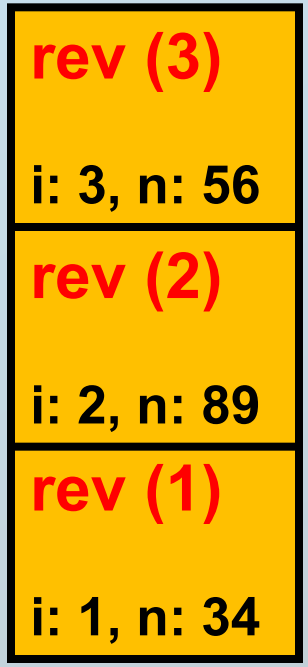
rev (1)

i: 1, n: 34

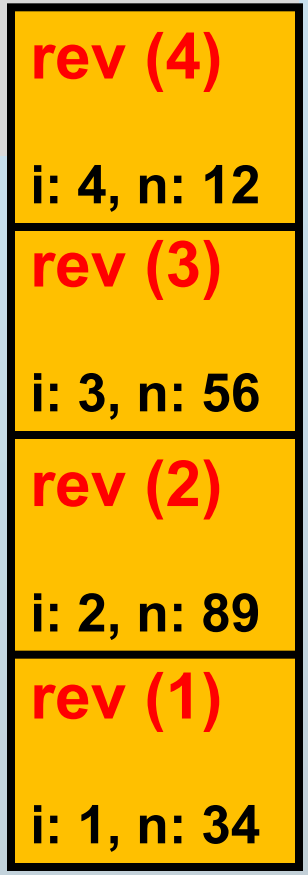
```
$ java Reverse
Enter the sequence of numbers to be reversed,
terminating with 0
Enter number 1: 34
Enter number 2: 89
```



```
$ java Reverse
Enter the sequence of numbers to be reversed,
terminating with 0
Enter number 1: 34
Enter number 2: 89
Enter number 3: 56
```

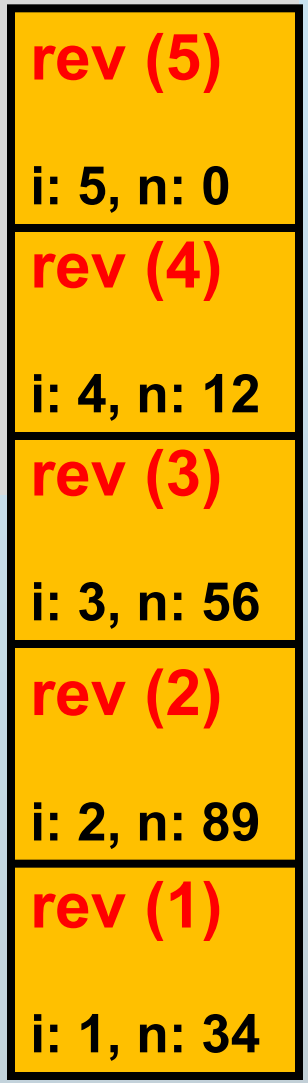


```
$ java Reverse
Enter the sequence of numbers to be reversed,
terminating with 0
Enter number 1: 34
Enter number 2: 89
Enter number 3: 56
Enter number 4: 12
```



```
$ java Reverse
Enter the sequence of numbers to be reversed,
terminating with 0
Enter number 1: 34
Enter number 2: 89
Enter number 3: 56
Enter number 4: 12
Enter number 5: 0
The numbers in reverse order are:
```

System Stack

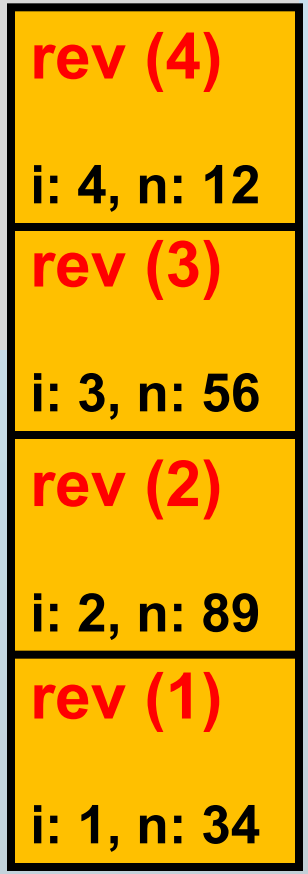


Pushing in function calls

```
$ java Reverse
Enter the sequence of numbers to be reversed,
terminating with 0
Enter number 1: 34
Enter number 2: 89
Enter number 3: 56
Enter number 4: 12
Enter number 5: 0
The numbers in reverse order are:
Number 4 is 12
```

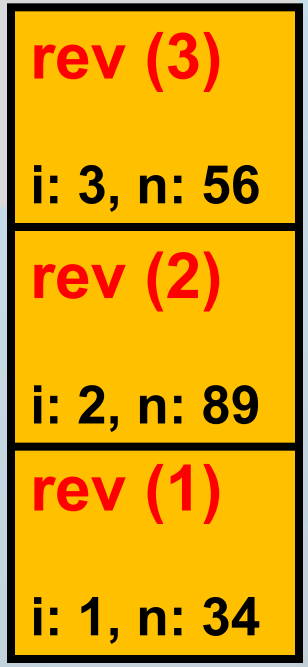
Popping the stack

System Stack



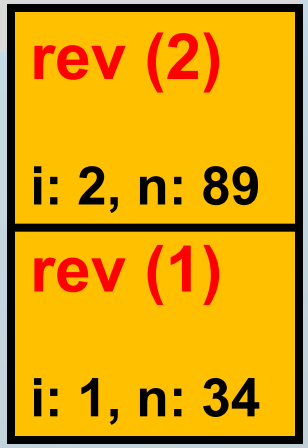
```
$ java Reverse
Enter the sequence of numbers to be reversed,
terminating with 0
Enter number 1: 34
Enter number 2: 89
Enter number 3: 56
Enter number 4: 12
Enter number 5: 0
The numbers in reverse order are:
Number 4 is 12
Number 3 is 56
```

System Stack



```
$ java Reverse
Enter the sequence of numbers to be reversed,
terminating with 0
Enter number 1: 34
Enter number 2: 89
Enter number 3: 56
Enter number 4: 12
Enter number 5: 0
The numbers in reverse order are:
Number 4 is 12
Number 3 is 56
Number 2 is 89
```

System Stack



```
$ java Reverse
Enter the sequence of numbers to be reversed,
terminating with 0
Enter number 1: 34
Enter number 2: 89
Enter number 3: 56
Enter number 4: 12
Enter number 5: 0
The numbers in reverse order are:
Number 4 is 12
Number 3 is 56
Number 2 is 89
Number 1 is 34
```

System Stack



```
$ java Reverse
Enter the sequence of numbers to be reversed,
terminating with 0
Enter number 1: 34
Enter number 2: 89
Enter number 3: 56
Enter number 4: 12
Enter number 5: 0
The numbers in reverse order are:
Number 4 is 12
Number 3 is 56
Number 2 is 89
Number 1 is 34
```

System Stack

Empty

```
// Enter a list of numbers interactively and store it
// incrementally and in reverse order in a table

public class Reverse {

  // Treating xs as a stack
  public static int[] add(int[] xs, int x) {
   if (xs==null){
      xs = new int[1];
      xs[0] = x;
    }
    else {
      int[] temp = xs;
      xs = new int[temp.length + 1];
      xs[0] = x;
      for (int i = 0; i < temp.length; i++)
          xs[i+1] = temp[i];
    }
    return xs;
  }
// continues . . . .
```

```
public static void main(String[] args) {

   int []xs = null;
   int num;
   do {
     System.out.print("Enter number: ");
     num = StdIn.readInt();
     if (num != 0) xs = add(xs,num);
   } while (num != 0);

   if (xs != null){
    System.out.println("The numbers in reverse order are:");
    for (int x : xs) System.out.println(x);
   }
 }

}
```

```
$ java Reverse
Enter number: 34
Enter number: 89
Enter number: 56
Enter number: 12
Enter number: 0
The numbers in reverse order are:
12
56
89
34
```

Είσοδος - Έξοδος
Με ανακατεύθυνση

- Το standard κανάλι εισόδου/εξόδου ανακατευθύνεται σε αρχείο κειμένου
  - Συντελείται σε επίπεδο λειτουργικού συστήματος
- Οι εντολές είναι οι ίδιες όπως για τη διαδραστική εισδοχή στοιχείων
  - Όμως, αφαιρούνται τα user prompts
  - Δεν εφαρμόζεται σε κάθε περίπτωση διαδραστικού προγράμματος (π.χ. «Είκοσι Ερωτήσεις»)
- Μόνο ένα αρχείο για την ανακατεύθυνση εισόδου/εξόδου
- Απεριόριστος αριθμός στοιχείων εισόδου
- Αποθήκευση αποτελέσματος για μακροπρόθεσμη αξιοποίηση

```
// Computing the average of a list of numbers
// Version 3: File redirection for input

public class Average {

 public static void main(String[] args){
  double sum = 0.0;
  int cnt = 0;
  System.out.print("Enter value: ");
  while (!StdIn.isEmpty()){
     sum +=  StdIn.readDouble();
     cnt++;
     System.out.print("Enter value: ");
  }
  double average = sum/cnt;
  System.out.println("\nAverage is " + average);
 }

}
```

```
// Computing the average of a list of numbers
// Version 3: File redirection for input

public class Average {

 public static void main(String[] args) {
  double sum = 0.0;
  int cnt = 0;
  while (!StdIn.isEmpty()){
     sum +=  StdIn.readDouble();
     cnt++;
  }
  double average = sum/cnt;
  System.out.println("\nAverage is " + average);
 }
}
```

```
Data.txt

45 67
2
109 200
```

```
$ java Average < Data.txt
Average is 84.6
```

```
// Generates a random sequence of N numbers from
// base to limit inclusive.
// N, base and limit entered from command line

public class RandomSeq {

 public static void main(String[] args) {
  int N = Integer.parseInt(args[0]);
  int base = Integer.parseInt(args[1]);
  int limit = Integer.parseInt(args[2]);
  for (int i = 1; i <= N; i++){
    System.out.print(base +
        (int)(Math.random()*(limit-base+1)) + "  ");
    if (i % 10 == 0) System.out.println();
  }
 }

}
```

```
$ java RandomSeq 20 1 10
6  6  5  2  2  9  9  7  8  10
6  5  4  7  3  8  1  6  8  7
```

Input από γραμμή εντολής και χρήση ανακατεύθυνσης για output

```
$ java RandomSeq 50 20 40 > Data.txt
$ cat Data.txt
35  36  39  23  32  25  38  30  25  39
28  35  30  39  32  35  24  23  25  28
28  30  25  25  37  40  21  35  34  28
22  40  29  40  31  23  27  38  31  30
31  31  29  28  36  40  31  25  32  36

$ java Average < Data.txt

Average is 31.08
```

Input μέσω ανακατεύθυνσης

Σημ: Η RandomSeq παράγει τους τυχαίους αριθμούς ως ακεραίους και η Average τους επεξεργάζεται ως πραγματικούς – Δεν υπάρχει κανένα πρόβλημα, το αντίθετο όμως θα ήταν πρόβλημα

Σωλήνωση Διεργασιών (process pipe)

$ process1 | process2 | process3

- Όπως και η ανακατεύθυνση, η «σωλήνωση» διεργασιών συντελείται σε επίπεδο λειτουργικού συστήματος
- Το output μίας διεργασίας, δηλαδή του υπό εκτέλεση προγράμματος, διοχετεύεται ως το input μιας άλλης διεργασίας, δηλαδή ενός άλλου υπό εκτέλεση προγράμματος
- Έτσι συντίθεται μια ακολουθία διεργασιών (με άλλα λόγια μέσω της ένωσής τους δημιουργείται ένας «σωλήνας» διεργασιών)
  - Μπορεί να προκύπτει και παράλληλη επεξεργασία, όπου δεν χρειάζεται να ολοκληρωθεί μια διεργασία για να αρχίσει η εκτέλεση της επόμενης στον σωλήνα διεργασίας



Επομένως το λογισμικό μας μπορεί να διασπαστεί σε επιμέρους προγράμματα τα οποία ορίζονται ως ξεχωριστές κλάσεις και που θα μπορούσαν να χρησιμοποιηθούν το καθένα ανεξάρτητα των άλλων:

- Άρθρωση σε υψηλό επίπεδο
- Πολλαπλές συνθέσεις διεργασιών
- Και κατ' επέκταση επαναχρησιμοποίηση

```
$ java RandomSeq 50 20 40 > Data.txt

$ java Average < Data.txt

Average is 31.08
```

```
$ java RandomSeq 50 20 40 | java Average

Average is 31.08
```

```
$ java RandomSeq 1000000 1 100 | java Average

Average is 50.99188678931984
```

Δεν υπάρχει το διαλογικό στοιχείο

Τεχνική της Σωλήνωσης

- Ιδιαίτερα κατάλληλη για προβλήματα που συνεπάγονται τον μετασχηματισμό του αρχικού input (εισόδου) σε διάφορες ενδιάμεσες μορφές μέχρι τον τελικό μετασχηματισμό που συνιστά το τελικό output (έξοδο)
- Ο σχεδιασμός αποτελείται από το αρχικό, τα ενδιάμεσα και το τελικό στάδιο της όλης διεργασίας

input.txt

```
Welcome to the website of
the Department of Computer
Science at the University of
Cyprus. The Department has
experienced significant growth
over the last few years,
and is now home to 22 faculty,
with strong groups in artificial
intelligence, computer architecture,
computer networks, data management,
graphics and virtual reality,
internet computing, software
engineering and theoretical
computer science.
```

output.txt

```
WELCOME TO THE WEBSITE OF

THE DEPARTMENT OF COMPUTER

SCIENCE AT THE UNIVERSITY OF

CYPRUS. THE DEPARTMENT HAS

EXPERIENCED SIGNIFICANT GROWTH

OVER THE LAST FEW YEARS,

AND IS NOW HOME TO 22 FACULTY,

WITH STRONG GROUPS IN ARTIFICIAL

INTELLIGENCE, COMPUTER ARCHITECTURE,

COMPUTER NETWORKS, DATA MANAGEMENT,

GRAPHICS AND VIRTUAL REALITY,

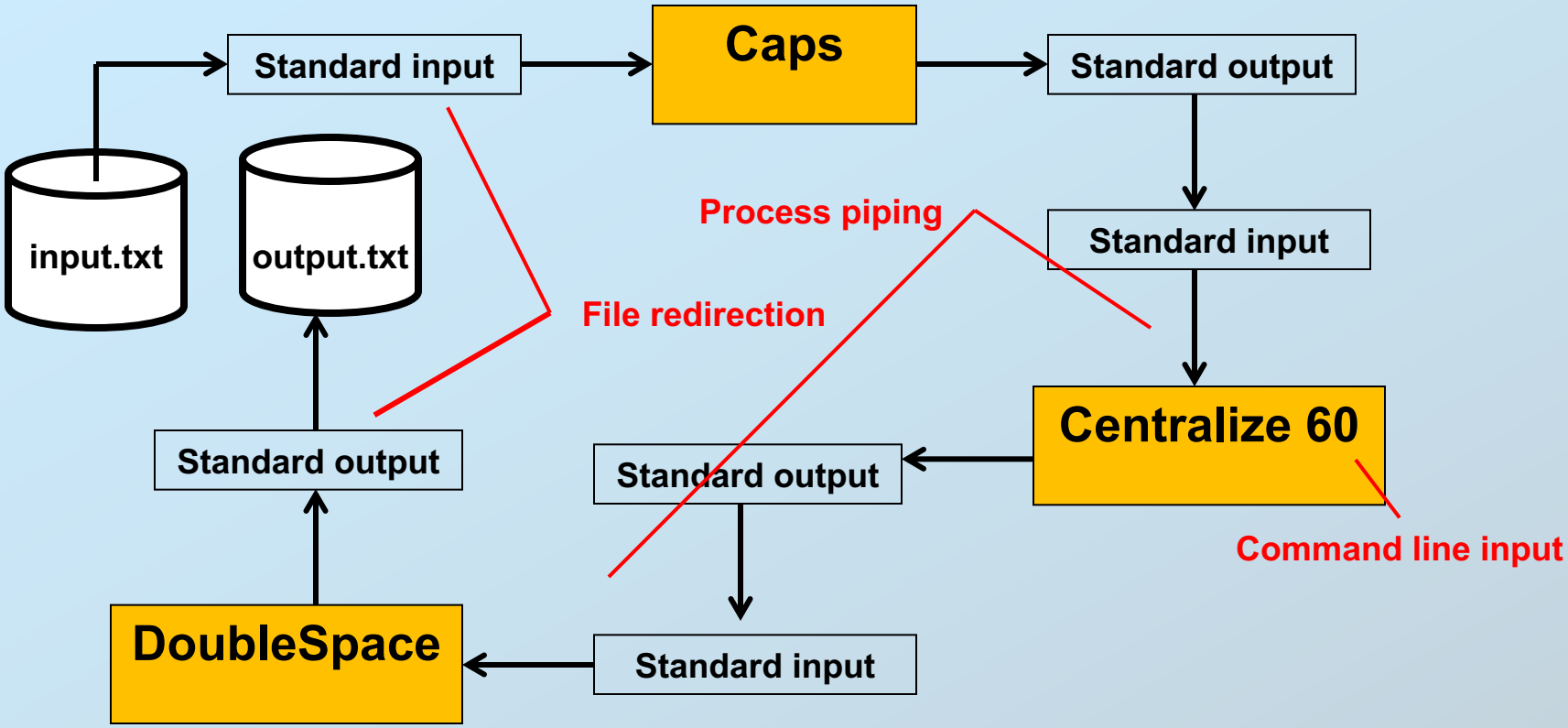
INTERNET COMPUTING, SOFTWARE

ENGINEERING AND THEORETICAL

COMPUTER SCIENCE.
```

Σωλήνωση Διεργασιών

```
$ java Caps < input.txt | java Centralize 60 | java DoubleSpace > output.txt
```



```
// Process Caps

public class Caps {

  private static char capL (char c) {
     if ('a' <= c && c <= 'z')
     return (char)(c - 'a' + 'A');
     else return c;
  }

  private static String convert(String s) {
   String rs = new String(); // = "";
   for (int i = 0; i < s.length(); i++)
      rs += capL(s.charAt(i));
   return rs;
  }

  public static void main (String [] args) {
    System.out.print(convert(StdIn.readAll()));
  }
}
```

```
// Process Centralize

public class Centralize {

 private static String blanks (int n) {
   String s = "";
   for (int i = 1; i <= n; i++) s += ' ';
   return s;
 }

 public static void main(String[] args) {
   int N = Integer.parseInt(args[0]);
   while (!StdIn.isEmpty()){
    String s = StdIn.readLine();
    int len = s.length();
    int left = (N - len)/2;
    int right = N - (len + left);
    System.out.print(blanks(left) + s +
      blanks(right) + "\n");
   }
 }
}
```

```
// Process DoubleSpace

public class DoubleSpace {

 public static void main(String[] args) {
   while(!StdIn.isEmpty()){
    String s = StdIn.readLine();
    System.out.print(s + "\n\n");
   }
 }

}
```

Σωλήνωση Διεργασιών

```
$ java Caps < input.txt | java Centralize 60 | java DoubleSpace > output.txt
```

Ποια θα είναι τα αποτελέσματα των ακόλουθων σωληνώσεων;

```
$ java Centralize 60 < input.txt | java DoubleSpace | java Caps > output.txt
```

```
$ java DoubleSpace < input.txt | java Caps | java Centralize 60 > output.txt
```

```
$ java Centralize 60 < input.txt | java Centralize 60 > output.txt
```

```
$ java Caps < input.txt | java Caps > output.txt
```

```
$ java DoubleSpace < input.txt | java DoubleSpace > output.txt
```

Επισήμανση αναφορικά με την Επαναχρησιμοποίηση

- Η επαναχρησιμοποίηση μέσω σωλήνωσης γίνεται σε επίπεδο κλάσεων, δηλαδή σε σχέση με την ολότητα της διεργασίας του κάθε προγράμματος.
  - Επί της ουσίας, η σωλήνωση αφορά τις συναρτήσεις main των εμπλεκόμενων κλάσεων/διεργασιών
- Βεβαίως τμήματα μίας κλάσης, δηλαδή ενός προγράμματος, θα μπορούσαν ενδεχομένως να αξιοποιηθούν (επαναχρησιμοποιηθούν) στο πλαίσιο κάποιου άλλου προγράμματος, νοουμένου ότι οι εν λόγω συναρτήσεις/μέθοδοι δηλώνονται ως public και όχι ως private (απόκρυψη πληροφοριών).
- Εξ ορισμού κλάσεις που συνιστούν Βιβλιοθήκες αποτελούνται από ένα σύνολο ξεχωριστών επαναχρησιμοποιήσιμων συναρτήσεων (π.χ. Math, StdIn, StdOut, System.out, κτλ.) και επομένως μια Βιβλιοθήκη δεν είθισται να αντιπροσωπεύει μια προγραμματιστική διεργασία με στόχο την επίλυση δεδομένου προβλήματος
  - Κατ' επέκταση κλάσεις που είναι Βιβλιοθήκες δεν εμπλέκονται σε σωληνώσεις διεργασιών
  - Εξάλλου η main μιας Βιβλιοθήκης είθισται να έχει ως σκοπό τη δοκιμή των υπολοίπων συναρτήσεων που περιλαμβάνονται στη Βιβλιοθήκη

Άσκηση: Να κατασκευαστεί η Βιβλιοθήκη TextProcess για την επεξεργασία κειμένου

```
public class TextProcess

-------------------------------------------------

      String Caps (String s)
      String Centralize (String s, int N)
      String DoubleSpace(String s)
```

```
// main in TextProcess used for testing the other functions

public static void main(String[] args){
   int N = Integer.parseInt(args[0]);
   System.out.print(DoubleSpace(Centralize(Caps(StdIn.readAll()), N)));}
```

```
// Outside TextProcess

System.out.print(TextProcess.DoubleSpace(
                    TextProcess.Centralize(
                        TextProcess.Caps(StdIn.readAll()), N)));
```

Ρητή Αναφορά σε Αρχεία

- Η ρητή αναφορά σε αρχεία συνεπάγεται την αξιοποίηση αφηρημένων τύπων για αρχεία εισόδου και αρχεία εξόδου:
  - Επομένως συγκεκριμένα αρχεία είναι περιπτώσεις **αντικειμένων**
  - Ως αφηρημένη έννοια, ένα αρχείο είναι απλά μια ακολουθία χαρακτήρων. Όμως, οι **μέθοδοι αναφοράς** επιτρέπουν την επεξεργασία ενός αρχείο σε υψηλότερο επίπεδο από αυτό των ατομικών χαρακτήρων
- Σε αντίθεση με την ανακατεύθυνση, τώρα μπορούμε να έχουμε ταυτόχρονα **πολλαπλά αρχεία** εισόδου ή/και εξόδου
  - Κανάλια εισόδου (input stream) και κανάλια εξόδου (output stream)

Κλάσεις Αντικειμένων `In` και `Out`

- Η κλάση **`In`** αντιπροσωπεύει αρχεία εισόδου
- Η κλάση **`Out`** αντιπροσωπεύει αρχεία εξόδου
- Η κλάση **`In`** περιλαμβάνει ταυτόσημες μεθόδους με την κλάση **`StdIn`** και παρομοίως η κλάση **`Out`** περιλαμβάνει ταυτόσημες μεθόδους με τις κλάσεις **`System.out`** και **`StdOut`**
- Όμως οι δικές τους μέθοδοι είναι **μέθοδοι αναφοράς** και όχι στατικές μέθοδοι όπως στην περίπτωση των κλάσεων **`StdIn`**, **`System.out`** και **`StdOut`**.
- Και βεβαίως οι κλάσεις **`In`** και **`Out`** περιλαμβάνουν τους ομώνυμους **κατασκευαστές** τους

Κλάση In

In(String name) **create input stream from name**

public class In		
boolean	isEmpty()	`true` *if no more values,* `false` *otherwise*
int	readInt()	*read a value of type* `int`
double	readDouble()	*read a value of type* `double`
long	readLong()	*read a value of type long*
boolean	readBoolean()	*read a value of type* `boolean`
char	readChar()	*read a value of type* `char`
String	readString()	*read a value of type* `String`
String	readLine()	*read the rest of the line*
String	readAll()	*read the rest of the text*

Κλάση Out

Out(String name) create output stream from name

public class Out	
void print(String s)	*print* s
void println(String s)	*print* s, *followed by newline*
void println()	*print a new line*
void printf(String f, ...)	*formatted print*

```
In ifile = new In("input.txt");
Out ofile = new Out("result.txt");

int count = 0;

while(!ifile.isEmpty()){
  char c = ifile.readChar();
  count++;
}

ofile.print(count);
ifile.close();
ofile.close();
```

```
int count = 0;

while(!StdIn.isEmpty()){
  char c = StdIn.readChar();
  count++;
}

System.out.print(count);
```

Πληροφορίες για In()

- `In()`
  - Χωρίς παραμέτρους επιστρέφει αναφορά στην προκαθορισμένη είσοδο
- `In(String)`
  - Ανοίγει το αρχείο με το όνομα της παραμέτρου
  - Παράδειγμα: προσπάθησε να ανοίξεις μια ιστοσελίδα
    - Πετάει σφάλμα/εξαίρεση (exception) αν δεν υπάρχει

```
String name = "https://introcs.cs.princeton.edu/java/home/";
In inf = new In(name);

System.out.println(name);

while(!inf.isEmpty())
    System.out.println(inf.readLine());
```

```
// Computing the average of a list of numbers
// Version 5: Reading input from file named
// on the command line

public class Average {

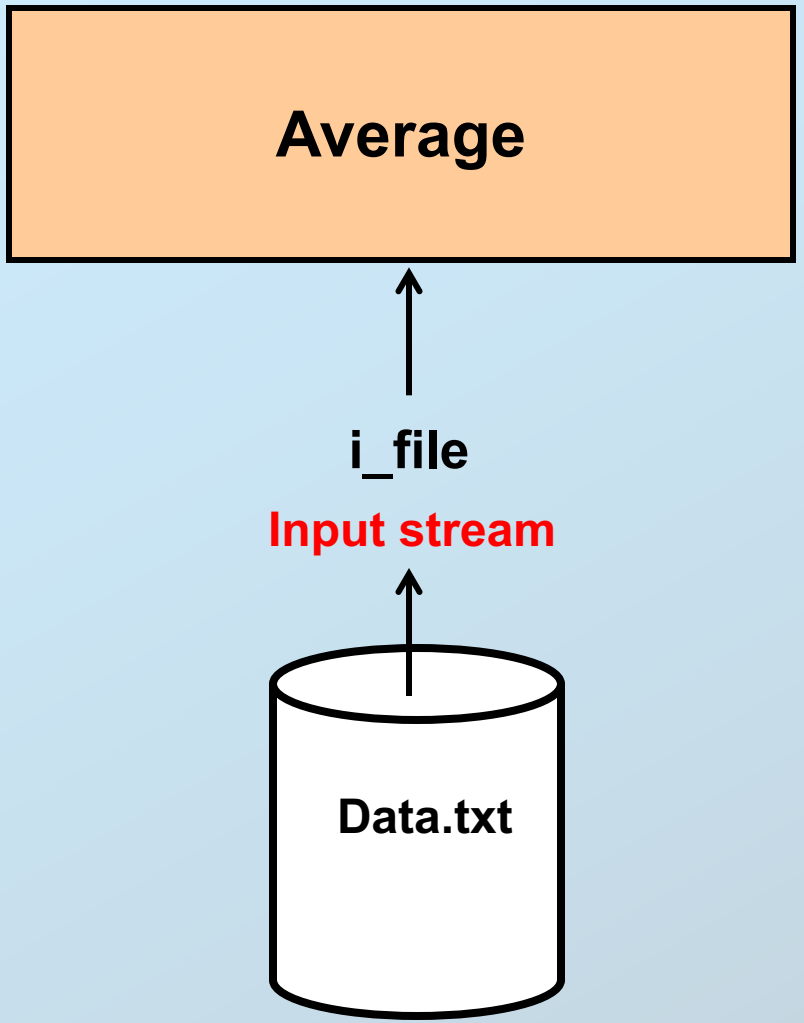
 public static void main(String[] args) {
  double sum = 0.0;
  int cnt = 0;
  In i_file = new In(args[0]);
  while (!i_file.isEmpty()){
     sum +=  i_file.readDouble();
     cnt++;
  }
  double average = sum/cnt;
  System.out.println("\nAverage is " + average);
 }

}
```

Data.txt

```
45 67
2
109 200
```

```
$ java Average Data.txt
Average is 84.6
```


Average
i_file
Input stream
Data.txt

```
// Appends the contents of a set of input files
// named on the command line.
// Files are sequentially processed.

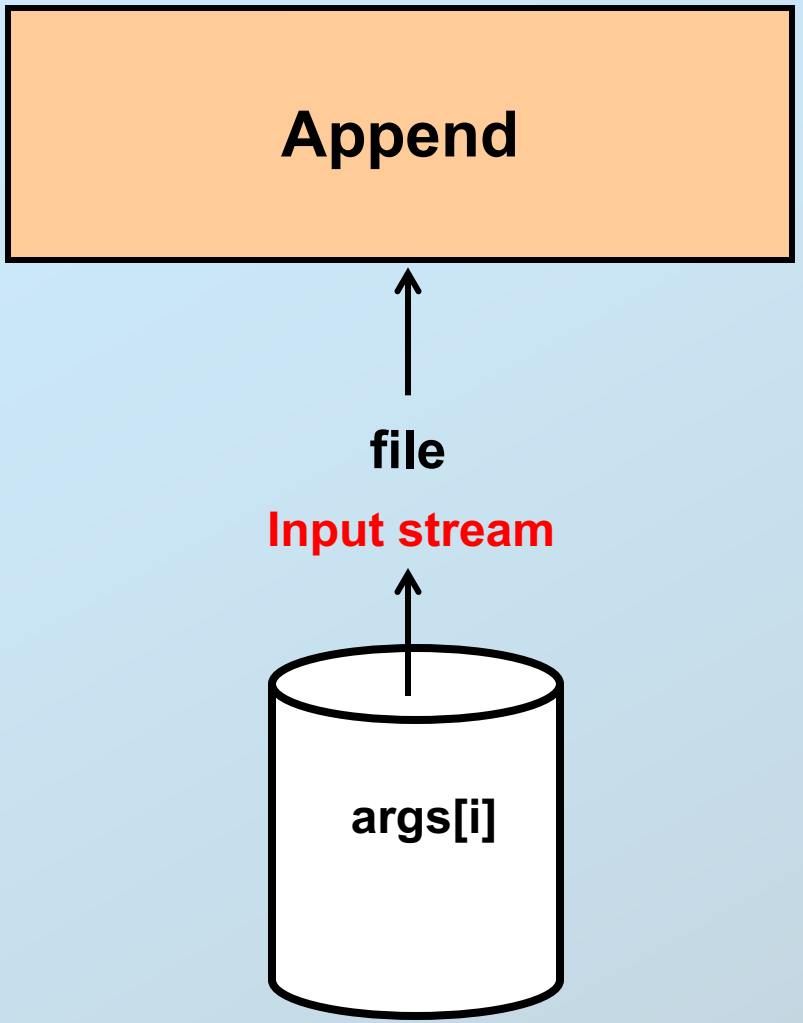
public class Append {

 public static void main(String[] args) {

  for (String name : args){
   In file = new In(name);
   String s = file.readAll();
   System.out.print(s);
   file.close(); // not necessary but good practice
  }

 }

}
```


Append
file
Input stream
args[i]

input1.txt

```
One two three
four five
six
eight nine
ten
```

input2.txt

```
1 2 3
4 5 6 8
9
```

file1.txt

```
1  19  16  14  3  2  18  19  20  1
```

file2.txt

```
38  34  39  36  32  39  40  38  31  36
40  31  36  38  39
```

file3.txt

```
8  10  10  8  9  9  6  7  8  10
6  8  7  8  9  9  10  10  7  6
```

```
$ java Append input1.txt input2.txt
One two three
four five
six
eight nine
ten
1 2 3
4 5 6 8
9
```

```
$ java Append input1.txt input2.txt > result.txt
$ cat result.txt
One two three
four five
six
eight nine
ten
1 2 3
4 5 6 8
9
```

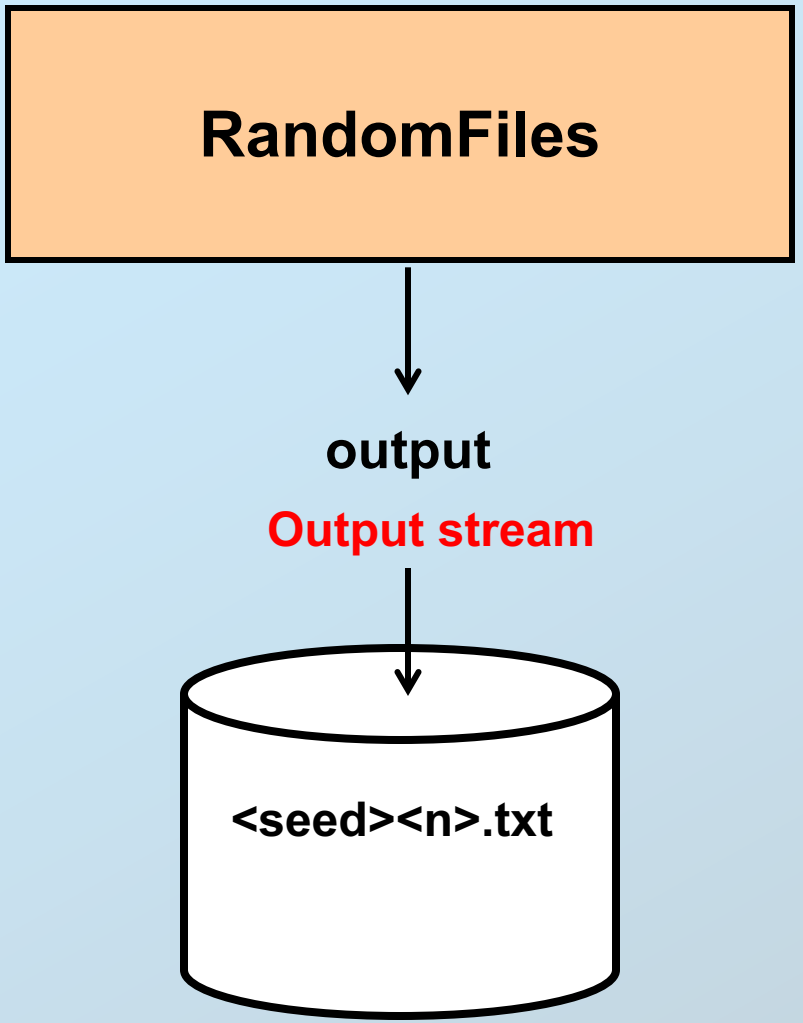
```
$ java Append file1.txt file2.txt file3.txt > result.txt
$ cat result.txt
1  19  16  14  3  2  18  19  20  1
38  34  39  36  32  39  40  38  31  36
40  31  36  38  39  8  10  10  8  9  9  6  7  8  10
6  8  7  8  9  9  10  10  7  6
```

```
$ java Append file*.txt > result.txt
$ cat result.txt
1  19  16  14  3  2  18  19  20  1
38  34  39  36  32  39  40  38  31  36
40  31  36  38  39  8  10  10  8  9  9  6  7  8  10
6  8  7  8  9  9  10  10  7  6
```

```
// Creates M separate files containing random numbers
// as for RandomSeq

public class RandomFiles {

 public static void main(String[] args) {
  int M = Integer.parseInt(args[0]);
  String seed = args[1];
  int N = Integer.parseInt(args[2]);
  int base = Integer.parseInt(args[3]);
  int limit = Integer.parseInt(args[4]);
  for (int f = 1; f <= M; f++){
   String name = seed + f + ".txt";
   Out output = new Out(name);
   for (int i = 1; i <= N; i++){
    output.print(base + (int)(Math.random()*(limit-base+1))
                      + "  ");
    if (i % 10 == 0) output.println();
   }
   output.close();
  }
 }
}
```

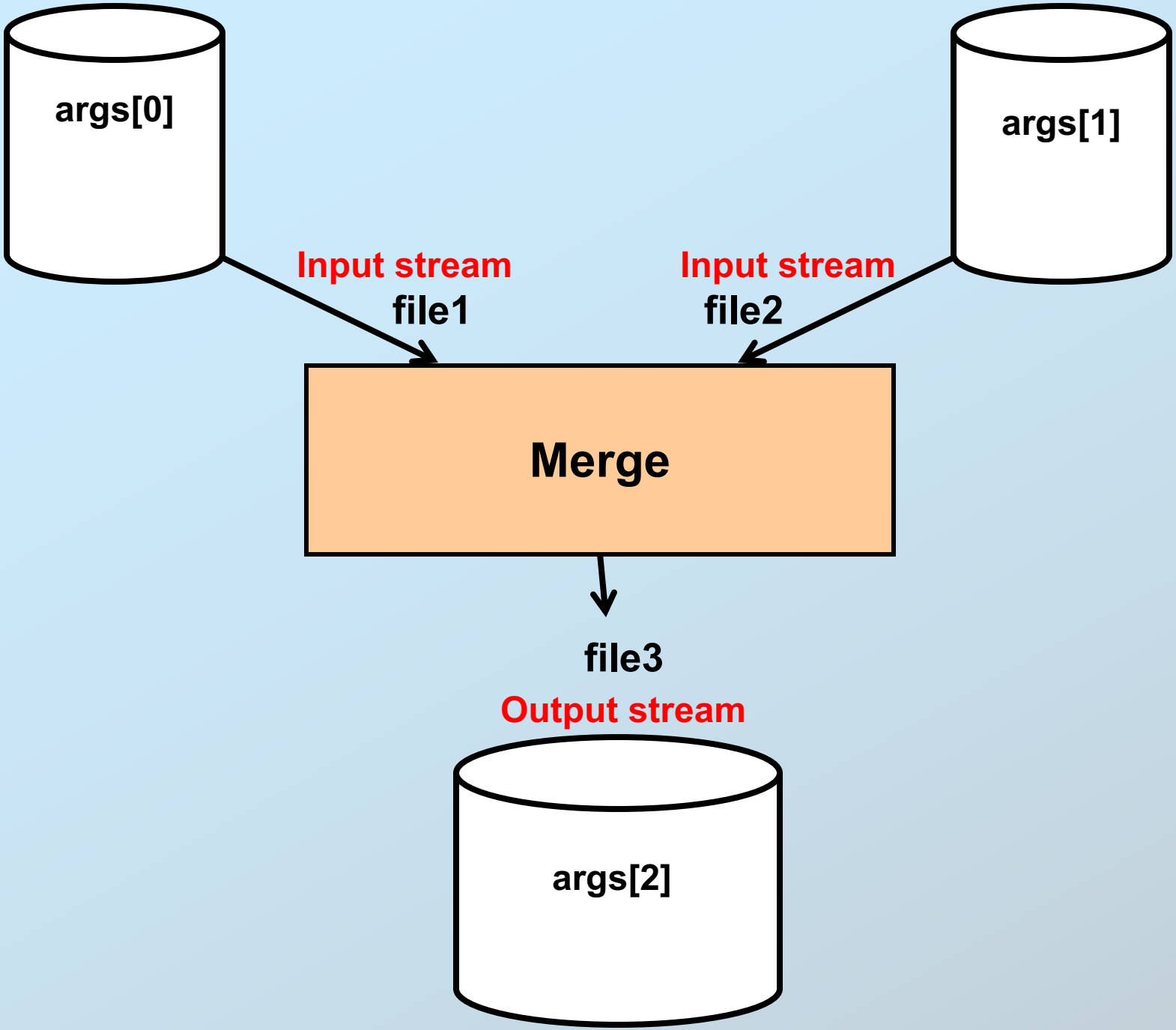

RandomFiles
output
Output stream
<seed><n>.txt

```
$ java RandomFiles 5 nums 10 1 100
$ ls nums*
nums1.txt  nums2.txt  nums3.txt  nums4.txt  nums5.txt
$ java Append nums* > result.txt
$ cat result.txt
80  34  13  31  1  96  56  79  33  12
68  24  37  7  55  8  90  88  31  96
93  89  54  54  84  11  22  8  90  38
44  41  97  28  60  23  45  85  42  20
14  28  73  86  55  74  70  81  71  8
```

```
// Merges the contents of two input files into an
// output file

public class Merge {

 public static void main (String[] args){
    In file1 = new In(args[0]);
    In file2 = new In(args[1]);
    Out file3 = new Out(args[2]);
    while (!file1.isEmpty() && !file2.isEmpty()){
      file3.println(file1.readLine());
      file3.println(file2.readLine());
    }
    if (!file1.isEmpty())
        file3.print(file1.readAll());
    if (!file2.isEmpty())
        file3.print(file2.readAll());
    file1.close();
    file2.close();
    file3.close();
 }
}
```


args[0]
args[1]
Input stream
file1
Input stream
file2
Merge
file3
Output stream
args[2]

input1.txt

```
One two three
four five
six
eight nine
ten
```

input2.txt

```
1 2 3
4 5 6 8
9
```

```
$ java Merge input1.txt input2.txt out.txt
$ cat out.txt
One two three
1 2 3
four five
4 5 6 8
six
9
eight nine
ten
```

```
// Concatenates a set of input files and appends
// them at the end of an (existing) output file

public class Concatenate {

 public static void main(String[] args) {
  In t_file = new In(args[0]);
  String s = t_file.readAll();
  t_file.close();
  Out o_file = new Out(args[0]);
  o_file.print(s);
  for (int i = 1; i < args.length; i++){
    In i_file = new In(args[i]);
    s = i_file.readAll();
    o_file.print(s);
    i_file.close();
  }
  o_file.close();
 }
}
```

out.txt

```
One two three
1 2 3
four five
4 5 6 8
six
9
eight nine
ten
```

file1.txt

```
1  19  16  14  3  2  18  19  20  1
```

file2.txt

```
38  34  39  36  32  39  40  38  31  36
40  31  36  38  39
```

file3.txt

```
8  10  10  8  9  9  6  7  8  10
6  8  7  8  9  9  10  10  7  6
```

```
$ java Concatenate out.txt file*txt
$ cat out.txt
One two three
1 2 3
four five
4 5 6 8
six
9
eight nine
ten
1  19  16  14  3  2  18  19  20  1
38  34  39  36  32  39  40  38  31  36
40  31  36  38  39  8  10  10  8  9  9  6  7  8  10
6  8  7  8  9  9  10  10  7  6
```

Επισημάνσεις για αρχεία κειμένου

- Το λειτουργικό σύστημα (Unix/Linux) δεν υποστηρίζει οποιαδήποτε δομή
  - Κάθε αρχείο κειμένου είναι απλά μια ακολουθία χαρακτήρων η οποία τερματίζεται από τον ειδικό χαρακτήρα EOF
- Η δομή ενός τέτοιου αρχείου είναι καθαρά «ιδεατή» και ερμηνεύεται μέσω των κατάλληλων εντολών εισόδου (για την εισδοχή στοιχείων από αρχείο κειμένου) ή εξόδου (για τη δημιουργία αρχείου κειμένου)
- Σε κάθε περίπτωση η επεξεργασία είναι σειριακή
- Τα αρχεία αποθηκεύονται σε δευτερεύουσα μνήμη
  - Για τη σχετική επεξεργασία τα περιεχόμενά τους πρέπει να εισαχθούν σε πρωτεύουσα μνήμη

Απλά Γραφικά για Visual Output

- Θα γίνει χρήση της Βιβλιοθήκης **`StdDraw`**
- Με απλά σχήματα μπορούν να δημιουργηθούν εντυπωσιακά γραφικά και να αποθηκευτούν σε δευτερεύουσα μνήμη (π.χ. αρχεία τύπου jpg) για μόνιμη αξιοποίηση
  - Σε τελική ανάλυση κάθε ψηφιακό/ή σχήμα/εικόνα είναι ένα σύνολο σημείων (ή εικονοστοιχείων) ή ένα σύνολο γραμμών (όπου κάθε γραμμή ενώνει δύο σημεία)
  - Ανάλυση και σύνθεση
- Υπάρχει η δυνατότητα για χρώματα και κίνηση (animation)

StdDraw: Βιβλιοθήκη για την παρουσίαση αποτελεσμάτων με γραφικό τρόπο

public class StdDraw

void line (double x0, double y0, double x1, double y1)	
void point (double x, double y)	
void text (double x, double y, String s)	
void circle (double x, double y, double r)	
void filledCircle (double x, double y, double r)	
void square (double x, double y, double r)	
void filledSquare (double x, double y, double r)	
void polygon (double[ ] x, double[ ] y)	
void filledPolygon (double[ ] x, double[ ] y)	
void setXscale (double x0, double x1)	reset x range to (x_0, x_1)
void setYscale (double y0, double y1)	reset y range to (y_0, y_1)
void setPenRadius (double r)	set pen radius to r
void setPenColor (Color c)	set pen color to c
void setFont (Font f)	set text font to f
void setCanvasSize (int w, int h)	set canvas to w-by-h window

Συνέχεια Βιβλιοθήκης **StdDraw**

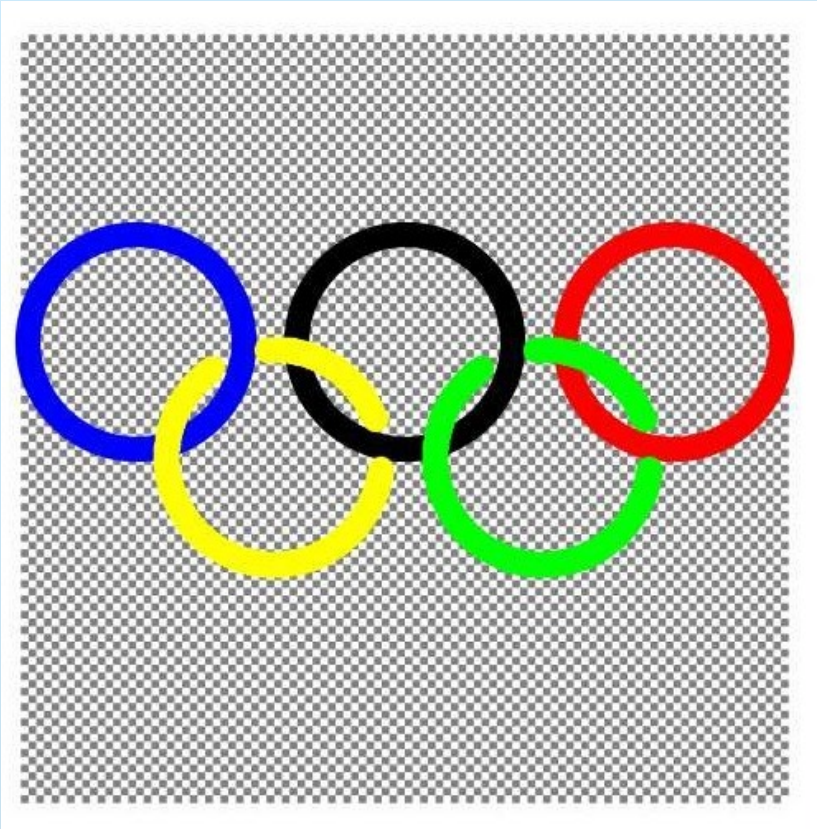
public class StdDraw

void clear (Color c)	clear the canvas; color it c
void show (int dt)	show all; pause dt milliseconds
void save (String filename)	save to a .jpg or w.png file

Στατικές Μέθοδοι

(όπως και για τις Βιβλιοθήκες StdIn, StdOut και System.out)

Παράδειγμα: Έμβλημα Ολυμπιακών Αγώνων



```
/* Πρόγραμμα που ζωγραφίζει το έμβλημα των
   Ολυμπιακών Αγώνων */

import java.awt.Color;

public class Olympic_Rings {

  public static void main(String[] args) {

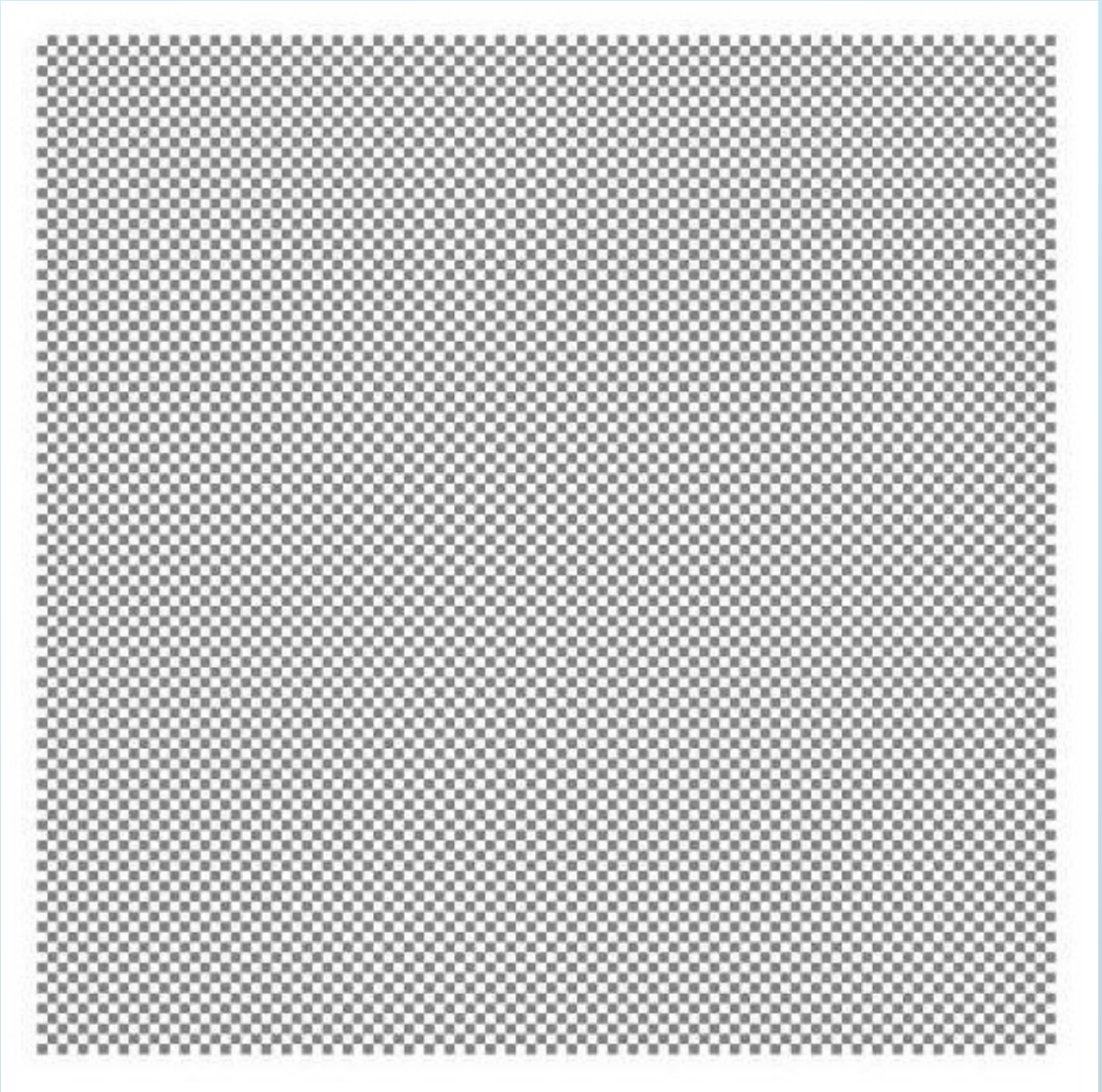
  }
}
```

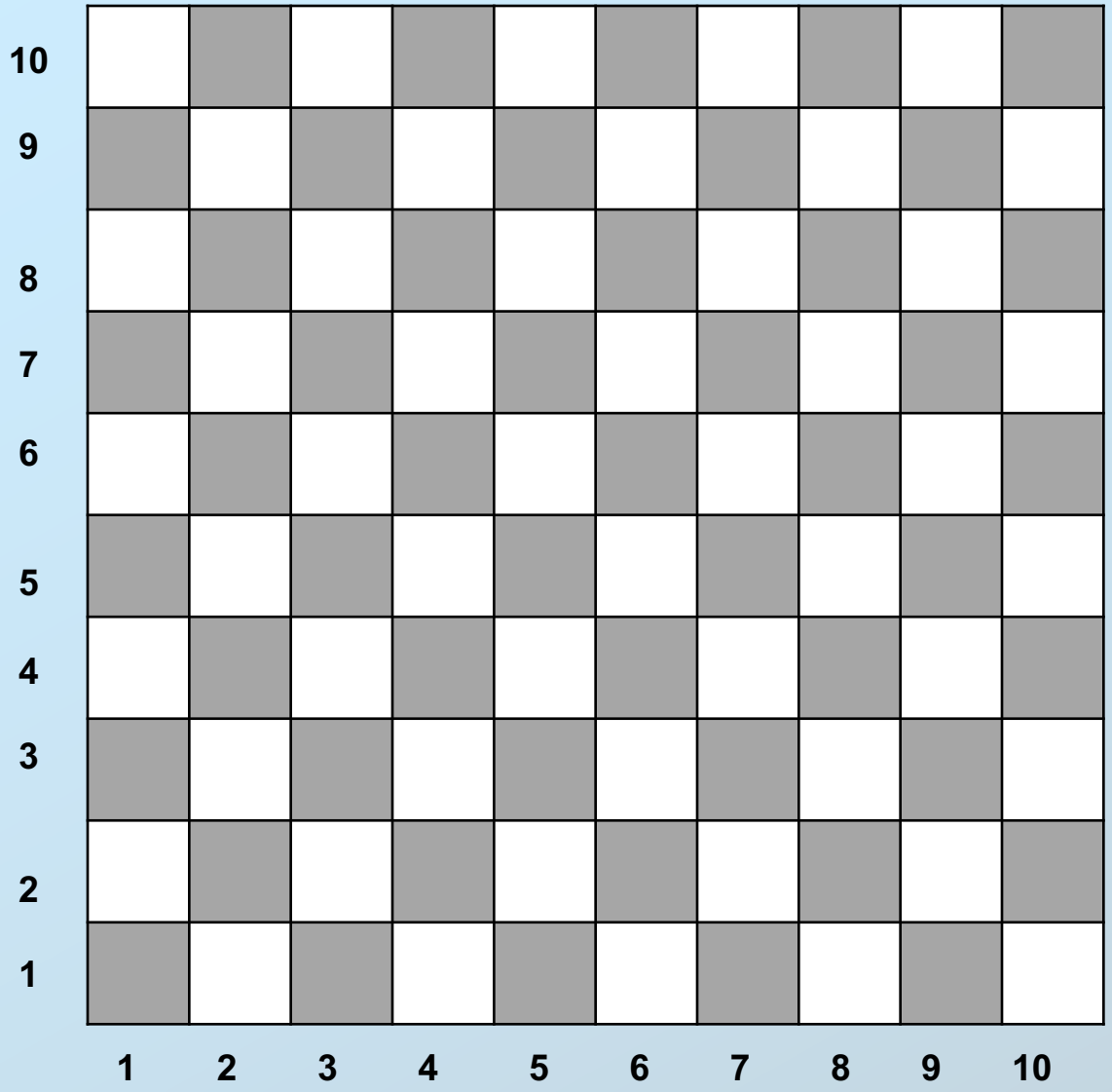
```
/* Βήμα 1: Προσδιορίζουμε το μέγεθος του πλαισίου (100 x 100)
           και δημιουργούμε το πλέγμα που αποτελεί το φόντο
           του εμβλήματος  */

    StdDraw.setXscale(0.0, 100.0);
    StdDraw.setYscale(0.0, 100.0);
    StdDraw.clear(Color.WHITE);

    StdDraw.setPenColor(Color.GRAY);

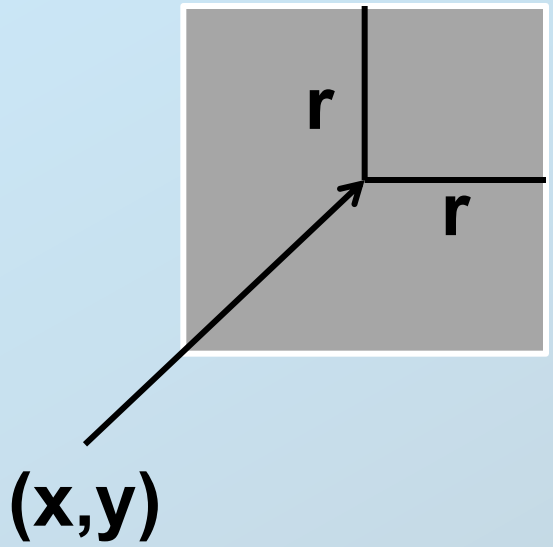
    . . . . . .
```





Τι παρατηρούμε;

- Το κάθε τετραγωνάκι προσδιορίζεται από τη σειρά (i) και τη στήλη (j) που βρίσκεται
- Για κάθε γκρίζο τετραγωνάκι
  - Είτε το i και το j είναι και οι δύο ζυγοί αριθμοί
  - Είτε είναι περιττοί αριθμοί



Ποια είναι η κατάλληλη αλγοριθμική δομή για τη δημιουργία των γκρίζων τετραγώνων;

<u>Διπλή απαριθμητή επανάληψη</u>

Επανέλαβε για κάθε σειρά i
    Επανέλαβε για κάθε στήλη j
        Εάν i και j είναι και οι δύο είτε ζυγοί,
            είτε περιττοί αριθμοί
                Ζωγράφισε στην εν λόγω θέση ένα
                γκρίζο τετραγωνάκι μεγέθους 1

```
for (int ri = 1; ri <= 100; ri++)                  // for every row ri
    for (int cj = 1; cj <= 100; cj++)              // for every col cj
        if ((ri % 2 == 1 && cj % 2 == 1) ||        // both odd?
            (ri % 2 == 0 && cj % 2 == 0))      // both even?
                StdDraw.filledSquare
                    ((ri - 1) + 0.5,  // x coordinate
                     (cj - 1) + 0.5,  // y coordinate
                      0.5);           // radius r
```

Γιατί το βήμα 1 (δημιουργία φόντου-πλέγματος) να μην κωδικοποιηθεί ως ξεχωριστή μέθοδος; Άρθρωση

```
public static void draw_background_grid() {

   StdDraw.setXscale(0.0, 100.0);
   StdDraw.setYscale(0.0, 100.0);
   StdDraw.clear(Color.WHITE);

   StdDraw.setPenColor(Color.GRAY);
   for (int ri = 1; ri <= 100; ri++)
        for (int cj = 1; cj <= 100; cj++)
             if ((ri % 2 == 1 && cj % 2 == 1) ||
                 (ri % 2 == 0 && cj % 2 == 0))
                     StdDraw.filledSquare
                       ((double)(ri - 1) + 0.5,
                       (double)(cj - 1) + 0.5,
                       0.5);
}
```

```
/* Βήμα 1: Προσδιορίζουμε το μέγεθος του πλαισίου (100 x 100)
           και δημιουργούμε το πλέγμα που αποτελεί το φόντο
           του εμβλήματος  */

   draw_background_grid();
```

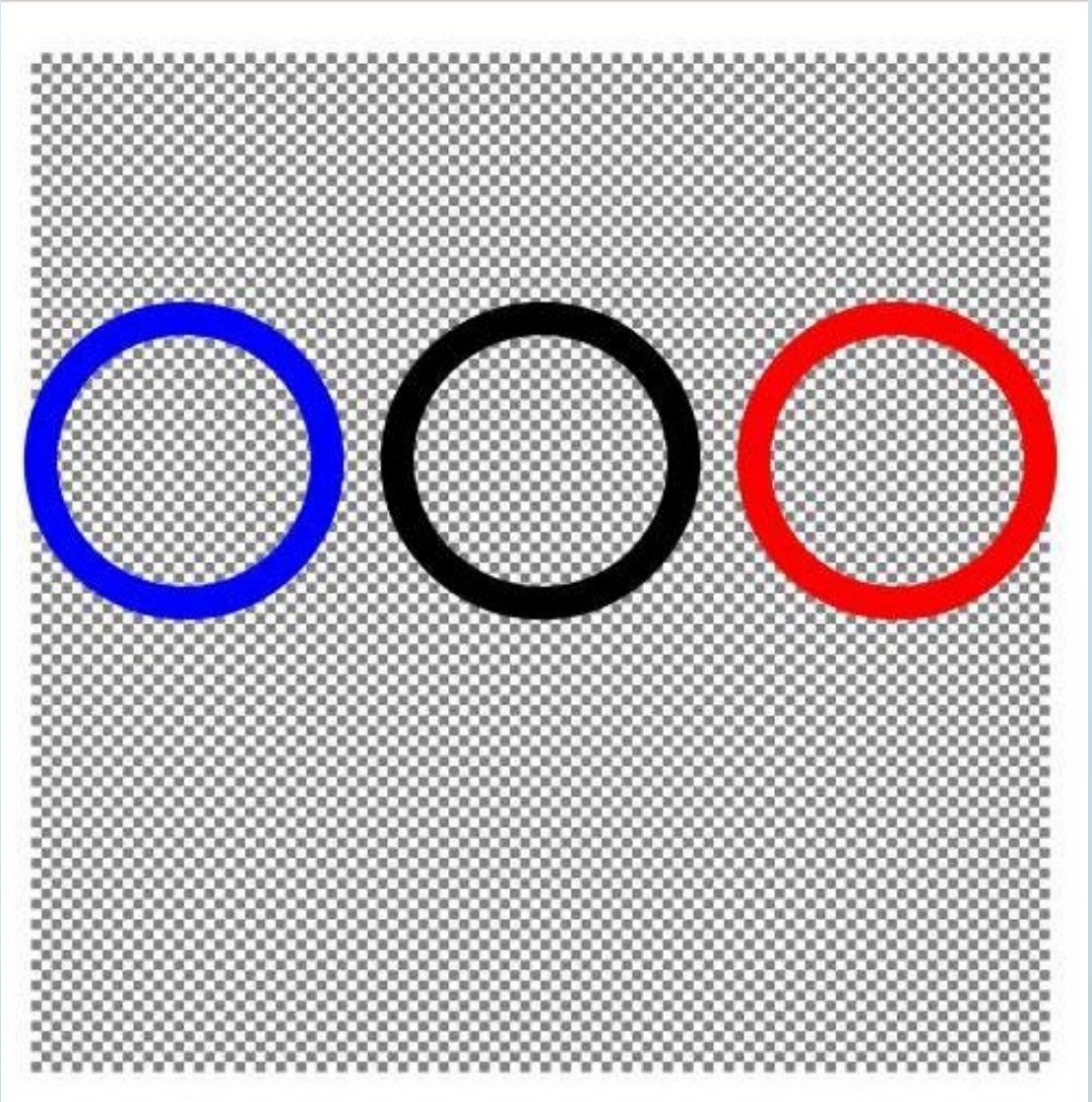
```
/* Βήμα 2: Ζωγραφίζουμε τους πάνω δακτύλιους*/

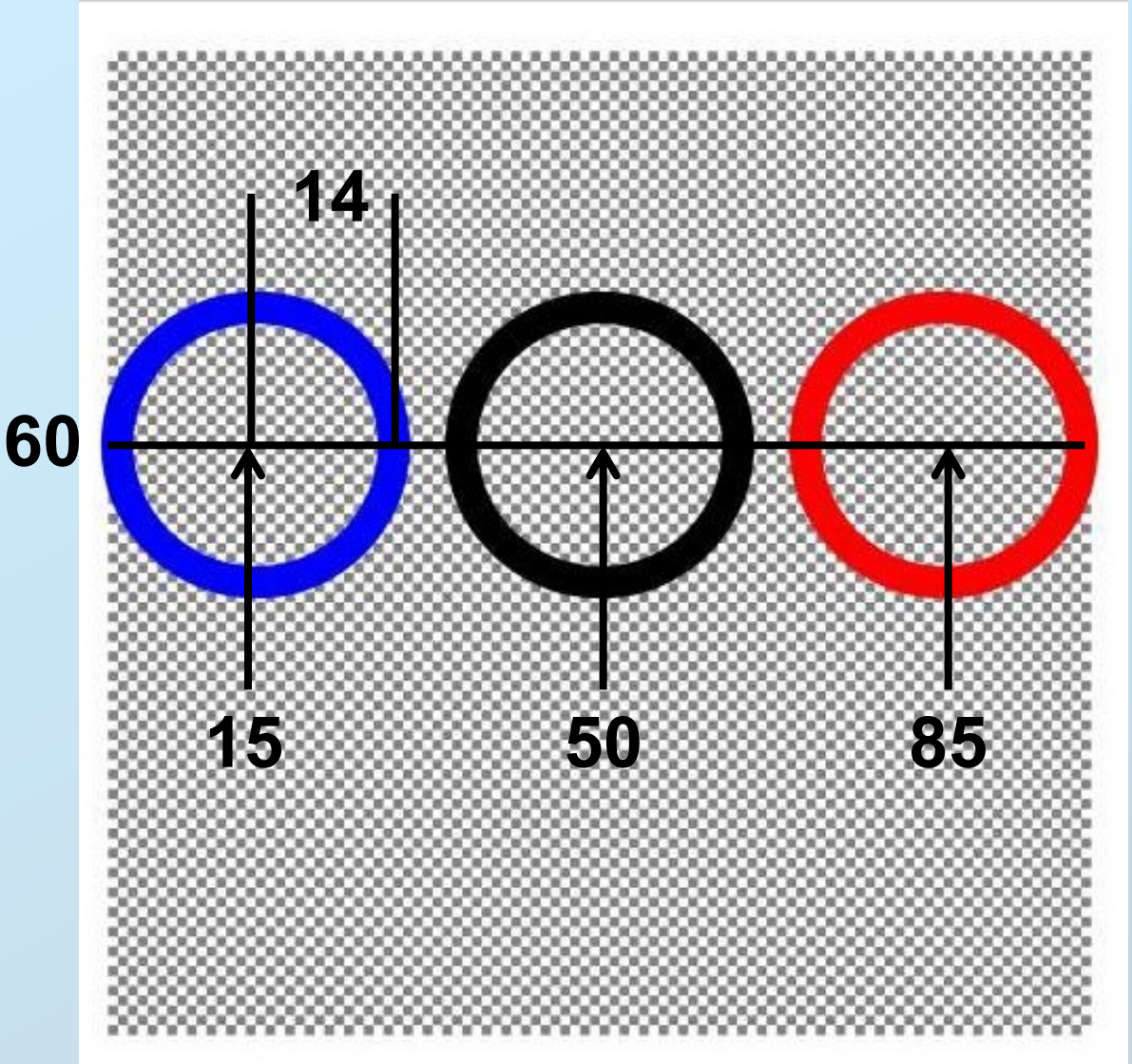
    StdDraw.setPenRadius(0.030);

    StdDraw.setPenColor(Color.BLUE);
    StdDraw.circle(15.0, 60.0, 14.0);

    StdDraw.setPenColor(Color.BLACK);
    StdDraw.circle(50.0, 60.0, 14.0);

    StdDraw.setPenColor(Color.RED);
    StdDraw.circle(85.0, 60.0, 14.0);
```

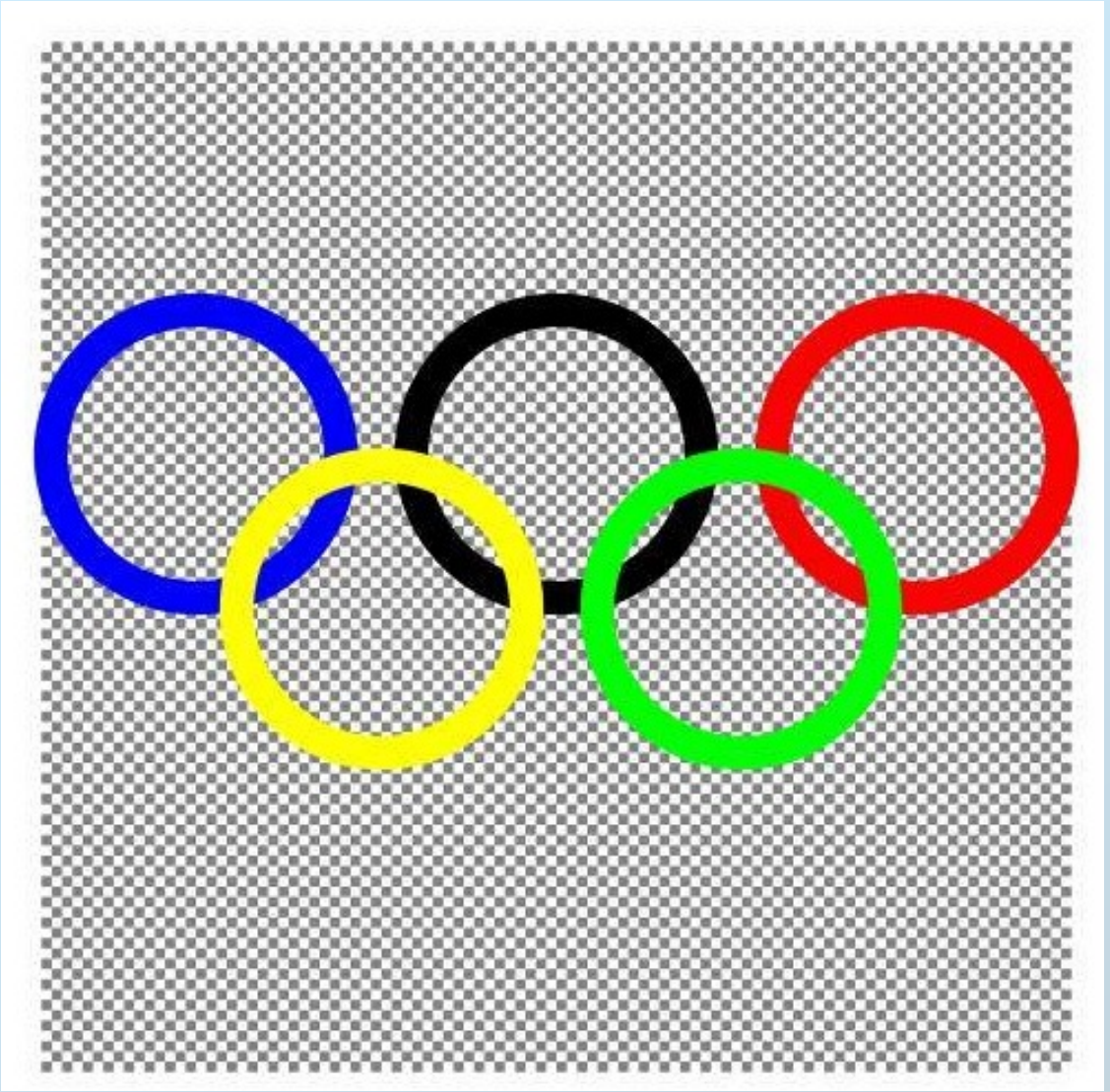


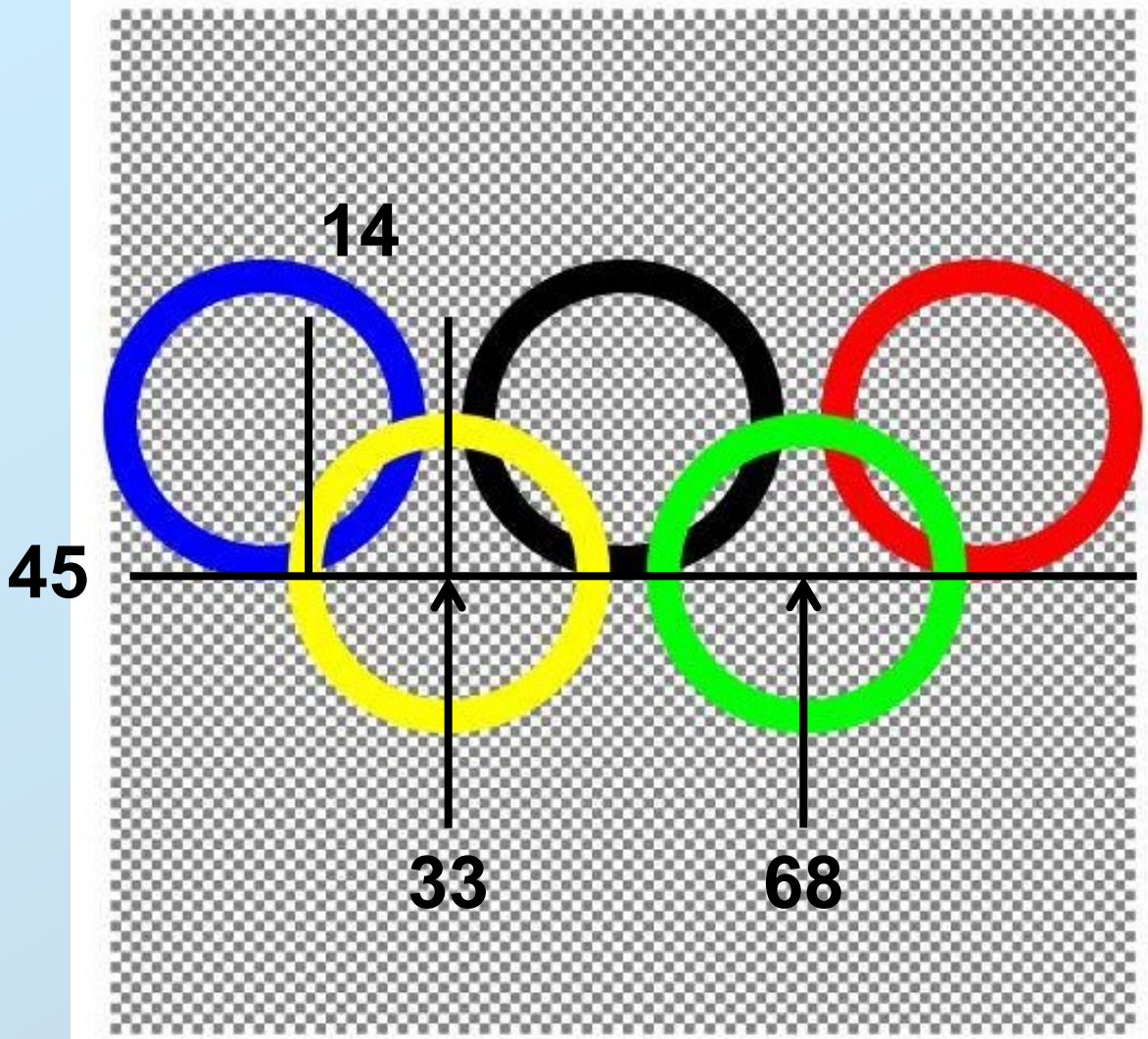

14
60
15
50
85

```
/* Βήμα 3: Ζωγραφίζουμε τους κάτω δακτύλιους */

    StdDraw.setPenColor(Color.YELLOW);
    StdDraw.circle(33.0, 45.0, 14.0);

    StdDraw.setPenColor(Color.GREEN);
    StdDraw.circle(68.0, 45.0, 14.0);
```




14
45
33
68

Σύνοψη

- Τα προγράμματα μας πρέπει να επικοινωνούν με τον έξω κόσμο
  - Πλήθος μονάδων εισόδου και εξόδου
  - Standard κανάλι εισόδου το πληκτρολόγιο και standard κανάλι εξόδου η οθόνη
  - Βιβλιοθήκες StdIn, StdOut, System.out
    - Στατικές Μέθοδοι
- Ορίσματα από γραμμή εντολής (ως strings)
  - Διασύνδεση main με λειτουργικό σύστημα
- Διαδραστική εισδοχή δεδομένων
  - Βιβλιοθήκη StdIn

Σύνοψη συνέχεια ..

- Ανακατεύθυνση standard input/output σε αρχεία
  - Σε επίπεδο λειτουργικού συστήματος
- Σωλήνωση διεργασιών
  - Σε επίπεδο λειτουργικού συστήματος
  - Αρχιτεκτονική στρώσεων
- Ρητή αναφορά σε αρχεία
  - Πολλαπλά κανάλια εισόδου/εξόδου ταυτόχρονα
  - Βιβλιοθήκες In/Out
    - Αντικείμενα – Μέθοδοι Αναφοράς

Σύνοψη συνέχεια ..

- Γραφικά για visual output
  - Βιβλιοθήκη StdDraw
  - Στατικές Μέθοδοι
  - Χρώματα και κίνηση
    - Ψηφιακά σχήματα/εικόνες ως σύνολα σημείων ή γραμμών

Σημ: Δεν ασχοληθήκαμε με το (σημασιολογικό και άλλο) έλεγχο εγκυρότητας δεδομένων (είτε αυτά προέρχονται από γραμμή εντολής, ή διαλογικά ή από αρχεία) – Σημαντικό ζήτημα σε ρεαλιστικά προγράμματα, ως προς την ανθεκτικότητά τους, κτλ.

Στο βιβλίο

- **Ενότητα 3.1**
- **Ενότητα 1.5 (γραφικά)**